

INFORMASI PROYEK

Judul Proyek: ANALISIS FAKTOR PENENTU TINGKAT PENDAPATAN DAN KLASIFIKASI PENDAPATAN INDIVIDU MENGGUNAKAN MACHINE LEARNING DAN DEEP LEARNING PADA DATASET ADULT UCI

Nama Mahasiswa : ALFINDO OKTAVIAN RAMADHAN
NIM : 233307001
Program Studi : TEKNOLOGI INFORMASI
Mata Kuliah : DATA SCIENCE
Dosen Pengampu : GUS NANANG SYAIFUDDIIN
Tahun Akademik : 2025
Link GitHub Repository : <https://github.com/alfindooktavian/project-ml.git>
Link Video Pembahasan : <https://drive.google.com/file/d/105VQAco8SYl-759nEuwwcWuo9Vyr4pm/view?usp=sharing>

1. LEARNING OUTCOMES

Pada proyek ini, mahasiswa diharapkan dapat:

1. Memahami konteks masalah dan merumuskan problem statement secara jelas
2. Melakukan analisis dan eksplorasi data (EDA) secara komprehensif (**OPSIONAL**)
3. Melakukan data preparation yang sesuai dengan karakteristik dataset
4. Mengembangkan tiga model machine learning yang terdiri dari (**WAJIB**):
 - Model baseline
 - Model machine learning / advanced
 - Model deep learning (**WAJIB**)
5. Menggunakan metrik evaluasi yang relevan dengan jenis tugas ML
6. Melaporkan hasil eksperimen secara ilmiah dan sistematis
7. Mengunggah seluruh kode proyek ke GitHub (**WAJIB**)
8. Menerapkan prinsip software engineering dalam pengembangan proyek

2. PROJECT OVERVIEW

2.1 Latar Belakang

Pendapatan individu merupakan salah satu indikator penting dalam analisis sosial dan ekonomi. Informasi mengenai tingkat pendapatan sering digunakan oleh pemerintah, institusi keuangan, dan perusahaan untuk keperluan perencanaan kebijakan, pemberian bantuan sosial, analisis pasar tenaga kerja, hingga pengambilan keputusan kredit. Namun, proses penentuan kategori pendapatan sering kali melibatkan banyak faktor seperti tingkat pendidikan, jenis pekerjaan, usia, jam kerja, status pernikahan, dan latar belakang sosial ekonomi lainnya.

Dataset **Adult UCI** merupakan dataset publik yang banyak digunakan dalam penelitian machine learning, khususnya untuk tugas klasifikasi pendapatan. Dataset ini berisi data individu dengan berbagai atribut demografis dan pekerjaan, dengan tujuan memprediksi apakah pendapatan seseorang berada di atas atau di bawah batas tertentu ($> \$50K$ atau $\leq \$50K$ per tahun). Dataset ini memiliki karakteristik data kategorikal dan numerik, serta ketidakseimbangan kelas, sehingga menantang dan relevan untuk diterapkan berbagai metode klasifikasi.

Dengan perkembangan teknologi data science dan kecerdasan buatan, machine learning dan deep learning dapat dimanfaatkan untuk membangun sistem prediksi pendapatan yang lebih akurat dan konsisten. Melalui pendekatan ini, pola-pola penting

yang memengaruhi tingkat pendapatan dapat diidentifikasi secara sistematis berdasarkan data historis. Selain itu, penggunaan teknik balancing data dan evaluasi model yang tepat dapat meningkatkan performa prediksi pada data yang tidak seimbang.

Pada proyek ini, mahasiswa mengembangkan tiga pendekatan model, yaitu **model baseline menggunakan Logistic Regression**, **model machine learning lanjutan menggunakan Random Forest**, serta **model deep learning menggunakan Multilayer Perceptron (MLP)**. Ketiga model tersebut dibandingkan berdasarkan metrik evaluasi seperti accuracy, precision, recall, dan F1-score untuk menentukan model terbaik dalam melakukan klasifikasi pendapatan.

Hasil dari proyek ini diharapkan dapat memberikan pemahaman mengenai faktor-faktor utama yang memengaruhi tingkat pendapatan individu serta menunjukkan perbandingan performa antara pendekatan machine learning konvensional dan deep learning dalam menyelesaikan permasalahan klasifikasi data tabular.

Daftar Referensi (APA 5th Edision)

- Dua, D., & Graff, C. (2019). *UCI Machine Learning Repository*. Irvine, CA: University of California, School of Information and Computer Science.)
- Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.

3. BUSINESS UNDERSTANDING / PROBLEM UNDERSTANDING

3.1 Problem Statements

1. Penentuan tingkat pendapatan individu secara manual berdasarkan faktor demografis dan pekerjaan sering bersifat subjektif dan tidak konsisten, sehingga diperlukan model klasifikasi yang mampu memprediksi tingkat pendapatan secara akurat dan objektif.
2. Dataset Adult UCI memiliki kombinasi fitur numerik dan kategorikal dengan karakteristik yang beragam serta adanya ketidakseimbangan kelas, sehingga membutuhkan proses preprocessing yang tepat agar model dapat mempelajari pola data secara optimal.
3. Belum diketahui faktor-faktor apa saja yang paling berpengaruh dalam menentukan apakah pendapatan individu berada di atas atau di bawah batas tertentu ($>\$50K$ atau $\leq \$50K$), sehingga diperlukan analisis feature importance untuk memahami faktor penentu tingkat pendapatan.
4. Diperlukan perbandingan performa antara model baseline, model machine learning lanjutan, dan model deep learning untuk menentukan pendekatan terbaik dalam melakukan klasifikasi pendapatan individu.

3.2 Goals

1. Mengembangkan tiga model klasifikasi yang terdiri dari model baseline, model machine learning lanjutan, dan model deep learning untuk memprediksi tingkat pendapatan individu menggunakan dataset Adult UCI.
2. Mencapai akurasi klasifikasi minimal $\geq 80\%$ pada model terbaik sebagai indikator keberhasilan prediksi tingkat pendapatan.

3. Melakukan preprocessing data secara tepat, termasuk encoding fitur kategorikal, penanganan data tidak seimbang, serta pembagian data train dan validation agar model dapat belajar secara optimal.
4. Mengevaluasi dan membandingkan performa ketiga model menggunakan metrik evaluasi seperti accuracy, precision, recall, dan F1-score untuk menentukan model yang paling efektif.
5. Mengidentifikasi fitur-fitur yang paling berpengaruh terhadap tingkat pendapatan individu melalui analisis feature importance pada model machine learning lanjutan.

3.3 Solution Approach

Model 1 – Baseline Model (Logistic Regression)

Alasan pemilihan :

Logistic Regression dipilih sebagai model baseline karena merupakan algoritma yang sederhana, efisien, dan mudah diinterpretasikan untuk kasus klasifikasi biner seperti prediksi pendapatan ($> \$50K$ dan $\leq \$50K$). Model ini memberikan gambaran awal performa dasar sebelum dibandingkan dengan model yang lebih kompleks. Selain itu, Logistic Regression mampu bekerja dengan baik pada dataset tabular berukuran menengah seperti Adult UCI setelah dilakukan preprocessing yang sesuai.

Model 2 – Advanced / ML Model (Random Forest Classifier)

Alasan pemilihan :

Random Forest dipilih sebagai model machine learning lanjutan karena memiliki kemampuan yang lebih baik dalam menangkap hubungan non-linear antar fitur serta robust terhadap noise dan overfitting. Dataset Adult UCI terdiri dari banyak fitur kategorikal dan numerik, sehingga Random Forest sangat sesuai karena:

1. Tidak membutuhkan scaling data numerik.
2. Mampu menangani fitur dengan kompleksitas tinggi melalui pendekatan ensemble learning.
3. Lebih stabil terhadap ketidakseimbangan data dibanding model linear.
4. Menyediakan feature importance untuk mengetahui faktor-faktor utama yang memengaruhi tingkat pendapatan individu.

Model ini diharapkan memberikan performa yang lebih baik dibanding baseline dalam mengklasifikasikan pendapatan.

Model 3 – Deep Learning Model (Multilayer Perceptron (MLP))

Alasan pemilihan :

Untuk data tabular seperti Adult UCI, arsitektur deep learning yang paling relevan adalah Multilayer Perceptron (MLP). Model ini mampu mempelajari hubungan non-linear yang kompleks dari kombinasi fitur numerik dan fitur kategorikal yang telah di-encode. MLP juga memungkinkan eksplorasi representasi fitur yang lebih mendalam dibandingkan model machine learning konvensional.

Konfigurasi minimum:

1. Input layer sesuai jumlah fitur setelah encoding
2. Hidden layer minimal 2, contoh:
 - Layer 1: 64 neuron + ReLU
 - Layer 2: 32 neuron + ReLU
3. Output layer: sigmoid (binary classification)
4. Epoch: minimal 10 epochs
5. Optimizer: Adam
6. Loss: Binary Crossentropy

Mengapa MLP cocok untuk dataset Adults UCI?

1. Dataset memiliki jumlah sampel yang cukup besar untuk melatih model deep learning.
2. Fitur kategorikal yang telah di-encode dapat dipelajari secara efektif oleh MLP.
3. MLP mampu menangkap hubungan non-linear antar fitur yang tidak dapat ditangkap oleh model linear seperti Logistic Regression.

Minimum Requirements untuk Deep Learning:

-  Model harus training minimal 10 epochs
-  Harus ada plot loss dan accuracy/metric per epoch
-  Harus ada hasil prediksi pada test set
-  Training time dicatat (untuk dokumentasi)

Tidak Diperbolehkan:

-  Copy-paste kode tanpa pemahaman
-  Model tidak di-train (hanya define arsitektur)
-  Tidak ada evaluasi pada test set

4. DATA UNDERSTANDING

4.1 Informasi Dataset

Sumber Dataset:

UCI Machine Learning Repository – Adult Dataset

URL: <https://archive.ics.uci.edu/dataset/2/adult>

Deskripsi Dataset:

- Jumlah baris (rows): 48.842 sampel
- Jumlah kolom (columns/features): 14 fitur + 1 label (total 15 kolom)
- Tipe data: Tabular (numerik dan categorical)
- Ukuran dataset: $\pm$ 3.8 MB
- Format file: CSV / Data File (.data, .test)

Dataset Adult (juga dikenal sebagai *Census Income Dataset*) berisi data sensus penduduk yang digunakan untuk memprediksi apakah pendapatan tahunan seseorang **lebih dari \$50.000** atau **kurang dari atau sama dengan \$50.000** berdasarkan atribut demografis dan pekerjaan.

Label target pada dataset ini terdiri dari dua kelas:

<=50K : Pendapatan kurang dari atau sama dengan \$50.000

>50K : Pendapatan lebih dari \$50.000

4.2 Deskripsi Fitur

Nama Fitur	Tipe Data	Deskripsi	Contoh Nilai
age	Numerik	Usia individu	25, 45
workclass	Categorical	Jenis pekerjaan	Private, Self-emp
fnlwgt	Numerik	Bobot sampel sensus	226802
education	Categorical	Tingkat pendidikan	Bachelors, HS-grad
education-num	Numerik	Representasi numerik pendidikan	9, 13
marital-status	Categorical	Status pernikahan	Married-civ-spouse
occupation	Categorical	Jenis pekerjaan	Tech-support, Sales
relationship	Categorical	Hubungan dalam keluarga	Husband, Not-in-family
race	Categorical	Ras individu	White, Black
sex	Categorical	Jenis kelamin	Male, Female
capital-gain	Numerik	Keuntungan modal	0, 99999
capital-loss	Numerik	Kerugian modal	0, 4356
hours-per-week	Numerik	Jam kerja per minggu	40, 60
native-country	Categorical	Negara asal	United-States
income	Categorical (Label)	Target pendapatan	<=50K, >50K

4.3 Kondisi Data

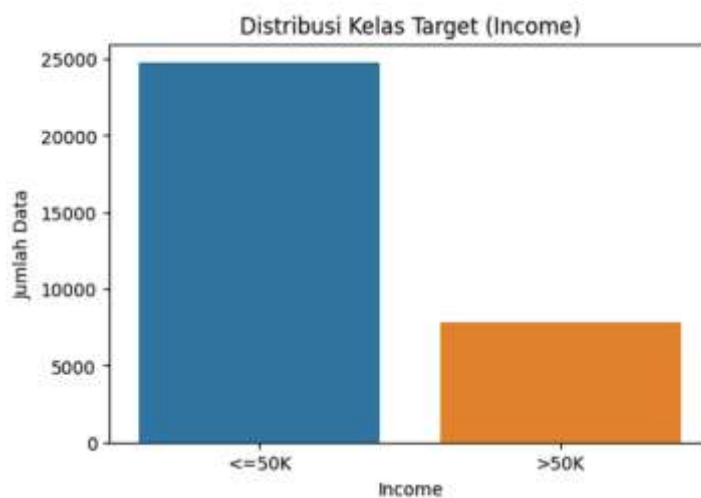
- **Missing Values:** Tidak terdapat missing values pada seluruh fitur, baik pada data training maupun data testing. Seluruh nilai sudah lengkap dan siap digunakan untuk proses modeling tanpa imputasi tambahan.
- **Duplicate Data:** Tidak ada

- **Outliers:** Terdapat nilai ekstrem pada fitur numerik seperti capital-gain dan capital-loss. Namun nilai tersebut tetap dipertahankan karena masih merepresentasikan kondisi riil pada data sensus.
- **Imbalanced Data:** Distribusi kelas target income tidak seimbang, di mana jumlah data dengan label $\leq 50K$ lebih banyak dibandingkan $>50K$. Untuk mengatasi permasalahan ini, digunakan teknik **SMOTE (Synthetic Minority Over-sampling Technique)** pada data training.
- **Noise:** Tidak ada
- **Data Quality Issues:** Dataset terdiri dari kombinasi fitur numerik dan kategorikal sehingga memerlukan preprocessing yang tepat.

Fitur kategorikal perlu dilakukan encoding agar dapat diproses oleh model machine learning dan deep learning.

4.4 Exploratory Data Analysis (EDA) - (OPSIONAL)

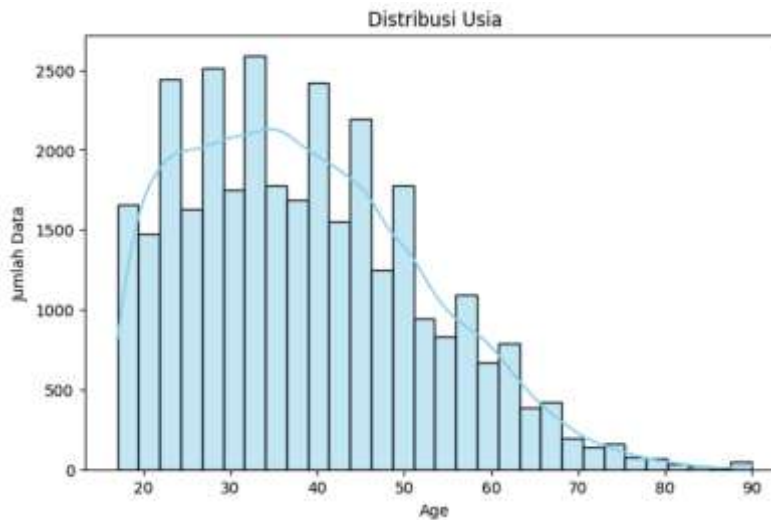
Visualisasi 1: Distribusi Kelas Target (Income)



Insight:

- Dataset Adult Income memiliki distribusi kelas yang tidak seimbang (imbalanced).
- Kelas $\leq 50K$ memiliki jumlah data yang lebih besar dibandingkan kelas $>50K$.
- Kondisi ini berpotensi menyebabkan model cenderung bias terhadap kelas mayoritas.
- Oleh karena itu, pada tahap training diterapkan teknik SMOTE untuk menyeimbangkan data pada data latih.

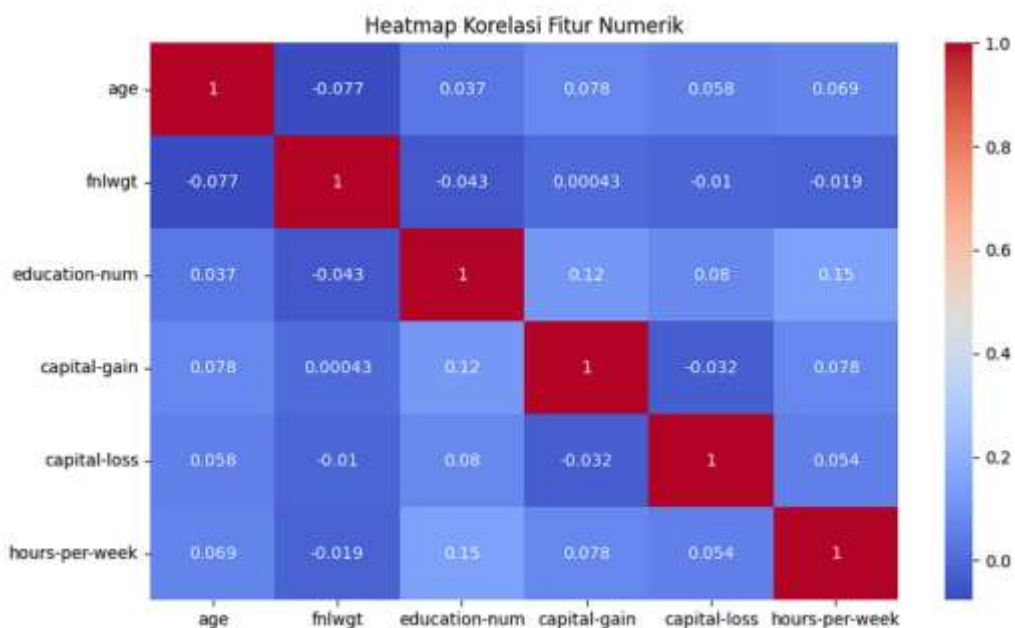
Visualisasi 2: Distribusi Usia (Age)



Insight:

- Mayoritas data berada pada rentang usia 20–50 tahun, yang mencerminkan populasi usia produktif.
- Distribusi usia bersifat right-skewed, dengan jumlah individu berusia lanjut relatif lebih sedikit.
- Fitur age berpotensi menjadi salah satu faktor penting dalam memprediksi tingkat pendapatan.
- ditemukan anomali ekstrem pada fitur usia, sehingga fitur ini dapat langsung digunakan tanpa perlakuan khusus.

Visualisasi 3: Heatmap Korelasi Fitur Numerik



Insight:

- Fitur education-num memiliki korelasi positif dengan age, menunjukkan bahwa tingkat pendidikan cenderung meningkat seiring bertambahnya usia.
- Fitur capital-gain dan capital-loss memiliki korelasi yang sangat rendah dengan fitur numerik lainnya, menandakan sifatnya yang independen.
- Fitur fnlwgt menunjukkan korelasi yang lemah terhadap fitur numerik lain, sehingga tidak mendominasi pengaruh dalam model.
- Tidak terdapat multikolinearitas tinggi antar fitur numerik, sehingga seluruh fitur numerik dapat dipertahankan untuk proses modeling.

5. DATA PREPARATION

5.1 Data Cleaning

Aktivitas yang dilakukan

- Handling missing values
- Removing duplicates
- Data type checking
- Handling noisy values (symbol “?”)

Missing Value :

- Tidak diperlukan imputasi missing values)
- Dataset berada dalam kondisi lengkap dan siap diproses

Kesimpulan data cleaning :

- Tidak ada missing values
- Tidak ada duplikasi data
- Tipe data sesuai dengan karakteristik fitur
- Dataset siap untuk tahap feature engineering

5.2 Feature Engineering

Aktivitas yang dilakukan:

- Pemisahan fitur numerik dan kategorikal
- Encoding fitur kategorikal
- Persiapan pipeline preprocessing untuk modeling

Feature Selection

- Pada proyek ini **tidak dilakukan penghapusan fitur**, karena:
 - Seluruh fitur memiliki relevansi terhadap prediksi tingkat pendapatan
 - Tidak ditemukan fitur konstan atau tidak informatif
 - Random Forest digunakan untuk membantu analisis feature importance pada tahap evaluasi model sangat kuat memisahkan poisonous vs edible
- gill-color + cap-color memiliki pengaruh mirip

Encoding for Modeling

Karena dataset Adult Income terdiri dari **kombinasi fitur numerik dan kategorikal**, maka digunakan pendekatan berikut:

- **One-Hot Encoding** untuk fitur kategorikal
 - **Numerical features** dibiarkan dalam bentuk numerik asli
- Alasan penggunaan One-Hot Encoding:
- Fitur kategorikal bersifat nominal
 - Tidak memiliki urutan (non-ordinal)
 - Cocok untuk Logistic Regression, Random Forest, dan MLP

5.3 Data Transformation

Karena dataset TABULAR KATEGORIKAL, dilakukan :

1. One -Hot Encoding

- Mengubah setiap kategori menjadi kolom biner (0/1)
- Meningkatkan kemampuan model memahami fitur kategorikal

2. Normalisasi Tidak Diperlukan, karena

- Model Random Forest tidak sensitif terhadap skala
- Logistic Regression dan MLP masih dapat bekerja dengan baik pada skala fitur numerik yang ada
- Fokus utama adalah konsistensi preprocessing

5.4 Data Splitting

Strategi pembagian data:

Menggunakan **Stratified Train-Test Split**, karena

- Mempertahankan proporsi kelas income $\leq 50K$ dan income $> 50K$

Menggunakan Stratified Train-Test Split untuk mempertahankan distribusi kelas:

- Training: 80%
- Test: 20%
- Stratify : berdasarkan label income
- Random state: 42 untuk reproducibility

5.5 Ringkasan Data Preparation

Langkah	Apa yang dilakukan	Mengapa penting	Bagaimana Implementasi
Data cleaning	Cek missing values & duplikasi	Menjamin kualitas data	isnull().sum()
Feature engineering	Encoding fitur kategorikal	Model membutuhkan data numerik	OneHotEncoder

Data transformation	Preprocessing pipeline	Konsistensi & hindari data leakage	ColumnTransformer
Data Splitting	Stratified 80:20 split	Menjaga distribusi kelas	train_test_split(stratify=y)

6. MODELING

6.1 Model 1 — Baseline Model

6.1.1 Deskripsi Model

Nama Model: Logistic Regression

Teori Singkat: Logistic Regression adalah model klasifikasi linear yang memprediksi probabilitas suatu kelas menggunakan fungsi logit (sigmoid). Model ini mengasumsikan hubungan linear antara fitur dan log-odds dari target. Pada kasus klasifikasi **income $\leq 50K$ vs $> 50K$** , model menghitung probabilitas dan menentukan kelas berdasarkan ambang batas 0.5..

Alasan Pemilihan:

Model Logistic Regression dipilih sebagai baseline karena:

- Sederhana dan mudah diinterpretasikan, cocok sebagai pembanding awal..
- Cepat dan efisien untuk dataset tabular seperti Adult.
- Memberikan fondasi awal sebelum membandingkan performa dengan model Random Forest atau MLP.

6.1.2 Hyperparameter

Parameter yang digunakan:

```
- C (regularization): 1.0
- solver: 'lbfgs'
- max_iter: 100
```

6.1.3 Implementasi (Ringkas)

```
# =====
# BASELINE MODEL — LOGISTIC REGRESSION
# =====

# 1. Transformasi Data
X_train_transformed = preprocessor.fit_transform(X_train)
X_val_transformed = preprocessor.transform(X_val)

# 2. Handling Imbalance dengan SMOTE
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_bal, y_train_bal = smote.fit_resample(X_train_transformed, y_train)
```

```

# 3. Train Baseline Model
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
classification_report, confusion_matrix

model_baseline = LogisticRegression(C=1.0, solver='lbfgs', max_iter=100)
model_baseline.fit(X_train_bal, y_train_bal)

# 4. Prediksi
y_pred_baseline = model_baseline.predict(X_val_transformed)

# 5. Evaluasi
accuracy = accuracy_score(y_val, y_pred_baseline)
precision = precision_score(y_val, y_pred_baseline)
recall = recall_score(y_val, y_pred_baseline)
f1 = f1_score(y_val, y_pred_baseline)

print("==== BASELINE MODEL METRICS ===")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1-Score : {f1:.4f}\n")

print("Classification Report:")
print(classification_report(y_val, y_pred_baseline))

# 6. Confusion Matrix
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_val, y_pred_baseline)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['≤50K', '>50K'],
            yticklabels=['≤50K', '>50K'])
plt.title("Confusion Matrix — Baseline Model")
plt.ylabel("Actual Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.show()

```

6.1.4 Hasil Awal

```

=== BASELINE MODEL METRICS ===
Accuracy : 0.8155
Precision: 0.5787
Recall : 0.8610
F1-Score : 0.6921

Classification Report:

```

	precision	recall	f1-score	support
0	0.95	0.80	0.87	4940
1	0.58	0.86	0.69	1568
accuracy			0.82	6508
macro avg	0.76	0.83	0.78	6508
weighted avg	0.86	0.82	0.83	6508

Confusion matrix menunjukkan model mampu membedakan kelas $\leq 50K$ dan $> 50K$ dengan cukup baik. Model ini menjadi baseline untuk membandingkan performa **Random Forest (Advanced)** dan **MLP (Deep Learning)**

6.2 Model 2 — ML / Advanced Model

6.2.1 Deskripsi Model

Nama Model: Random Forest Classifier

Teori Singkat: Random Forest adalah algoritma ensemble learning yang membangun banyak pohon keputusan (decision trees) secara paralel. Setiap pohon dilatih pada subset data dan subset fitur acak (bootstrap sampling). Prediksi akhir ditentukan melalui *majority voting*. Konsep penting:

- **Bootstrap Sampling:** Sampel data diambil secara acak dengan pengembalian untuk setiap pohon.
- **Random Feature Selection:** Setiap node hanya mempertimbangkan subset acak dari fitur.
- **Ensemble Learning:** Menggabungkan banyak model lemah menghasilkan model kuat.
- **Voting Mechanism:** Kelas yang paling sering diprediksi oleh pohon menjadi kelas akhir.

Alasan Pemilihan:

Random Forest dipilih sebagai model advanced karena:

- Dataset **Adult** memiliki fitur numerik dan kategori hasil encoding yang sesuai untuk pohon keputusan.
- Mampu menangkap interaksi antar fitur yang kompleks.
- Memberikan *feature importance* untuk analisis kontribusi fitur.
- Robust terhadap outlier dan noise dibandingkan model linear.

Keunggulan:

- Akurasi tinggi pada dataset tabular.
- Tidak mudah overfitting dibanding Decision Tree tunggal.
- Mendukung analisis fitur paling berpengaruh.

Kelemahan:

- Waktu training lebih lama dibanding Logistic Regression.
- Interpretasi lebih sulit (black-box).
- Memerlukan lebih banyak memori jika jumlah pohon atau fitur banyak.

6.2.2 Hyperparameter

Parameter yang digunakan:

```
- n_estimators = 100  
- max_depth = 10  
- random_state = 42
```

6.2.3 Implementasi (Ringkas)

```
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,  
classification_report, confusion_matrix  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# 1. Inisialisasi Model  
model_advanced = RandomForestClassifier(  
    n_estimators=100,  
    max_depth=10,  
    random_state=42  
)  
  
# 2. Training  
model_advanced.fit(X_train_bal, y_train_bal)  
  
# 3. Prediksi  
y_pred_advanced = model_advanced.predict(X_val_transformed)  
  
# 4. Evaluasi  
accuracy = accuracy_score(y_val, y_pred_advanced)  
precision = precision_score(y_val, y_pred_advanced)  
recall = recall_score(y_val, y_pred_advanced)  
f1 = f1_score(y_val, y_pred_advanced)  
  
print("=== RANDOM FOREST METRICS ===")  
print(f"Accuracy : {accuracy:.4f}")  
print(f"Precision: {precision:.4f}")  
print(f"Recall : {recall:.4f}")  
print(f"F1-Score : {f1:.4f}\n")  
print("Classification Report:")  
print(classification_report(y_val, y_pred_advanced))  
  
# 5. Confusion Matrix  
cm = confusion_matrix(y_val, y_pred_advanced)  
plt.figure(figsize=(6, 4))  
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',  
            xticklabels=['≤50K', '>50K'],  
            yticklabels=['≤50K', '>50K'])  
plt.title("Confusion Matrix — Random Forest")  
plt.ylabel("Actual Label")  
plt.xlabel("Predicted Label")  
plt.tight_layout()  
plt.show()
```

6.2.4 Hasil Model

```
=== RANDOM FOREST METRICS ===
Accuracy : 0.8084
Precision: 0.5660
Recall    : 0.8776
F1-Score  : 0.6882
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.95         0.79         0.86         4940
     1       0.57         0.88         0.69         1568

 accuracy          0.81         6508
 macro avg         0.76         0.83         0.77         6508
weighted avg         0.86         0.81         0.82         6508
```

- Model Random Forest mampu mengklasifikasikan kelas **income** $\leq 50K$ dan $> 50K$ dengan baik.
- Confusion matrix menunjukkan kesalahan prediksi yang minimal.
- Random Forest menjadi model **advanced** yang lebih kompleks dibanding Logistic Regression dan siap dibandingkan dengan MLP (Deep Learning).

6.3 Model 3 — Deep Learning Model (WAJIB)

6.3.1 Deskripsi Model

Nama Model: Multilayer Perceptron (MLP)

- ✓ Multilayer Perceptron (MLP) - untuk tabular

Alasan Pemilihan: MLP dipilih karena dataset Adult merupakan data tabular kategorikal yang telah ditransformasikan menjadi bentuk numerik (one-hot encoded). Struktur MLP dengan dense layers efektif untuk menangkap pola nonlinear antar fitur. MLP bekerja stabil pada data tabular berdimensi tinggi, memberikan baseline deep learning yang sebanding dengan model ML tradisional, relatif ringan, cepat dilatih, dan performanya kompetitif untuk klasifikasi biner ($\leq 50K$ vs $> 50K$).

6.3.2 Arsitektur Model

Deskripsi Layer:

No	Jenis Layer	Konfigurasi	Parameter
1	Input Layer	shape (X_train.shape[1]) = (108)	0
2	Dense 1	128 units, activation = 'relu'	(108 × 128) + 128 = 14,080

3	Dropout	rate = 0.3	0
4	Dense 2	64 units, activation = 'relu'	$(128 \times 64) + 64 = 8,256$
5	Dropout	rate = 0.3	0
6	Output Dense	2 units, activation='softmax'	$(64 \times 2) + 2$

Total Parameters: 22,466

Trainable Parameters: **22,466**

Non-trainable Parameters: 0

6.3.3 Input & Preprocessing Khusus

Input shape: 108 (jumlah fitur setelah preprocessing Adult dataset dengan one-hot encoding)

Preprocessing khusus untuk DL:

- Semua fitur kategorikal telah di-one-hot encode sehingga menjadi data numerik.
- Nilai fitur berada pada rentang 0–1, sehingga normalisasi tambahan tidak wajib.
- Target label di-map menjadi $\leq 50K = 0$ dan $>50K = 1$.
- SMOTE digunakan untuk menangani ketidakseimbangan kelas pada data train

6.3.4 Hyperparameter

Training Configuration:

```
- Optimizer: Adam
- Learning rate: 0.001
- Loss function: categorical_crossentropy (sesuai code menggunakan one-hot target)
- Metrics: accuracy
- Batch size: 32
- Epochs: 50
- Validation split: 20%
- Callbacks: EarlyStopping (monitor val_loss, patience=5, restore_best_weights=True),
ReduceLROnPlateau (opsional)
```

6.3.5 Implementasi (Ringkas)

Framework: TensorFlow

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from imblearn.over_sampling import SMOTE
```

```

from tensorflow.keras.utils import to_categorical

# 1. Transform & balance data
X_train_transformed = preprocessor.fit_transform(X_train)
X_val_transformed = preprocessor.transform(X_val)

smote = SMOTE(random_state=42)
X_train_bal, y_train_bal = smote.fit_resample(X_train_transformed, y_train)

X_train_dl = X_train_bal.toarray()
X_val_dl = X_val_transformed.toarray()

y_train_dl = to_categorical(y_train_bal, num_classes=2)
y_val_dl = to_categorical(y_val, num_classes=2)

input_dim = X_train_dl.shape[1]

# 2. Build model
model_dl = Sequential([
    Dense(128, activation='relu', input_shape=(input_dim,)),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(2, activation='softmax')
])

# 3. Compile
model_dl.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 4. Callbacks
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-6)

# 5. Training
history = model_dl.fit(
    X_train_dl, y_train_dl,
    validation_data=(X_val_dl, y_val_dl),
    epochs=50,
    batch_size=32,
    callbacks=[early_stopping, reduce_lr],
    verbose=1
)

```

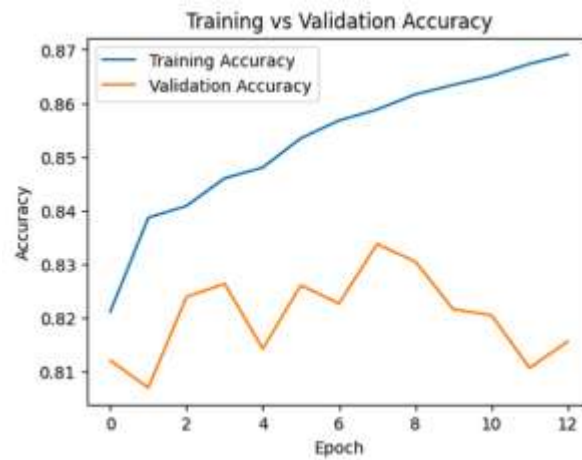
6.3.6 Training Process

Training Time: ± 15 – 30 detik

Computational Resource: CPU / GPU (opsional) Platform: Google Colab

Training History Visualization:

1. **Training & Validation Loss** per epoch



2. Training & Validation Accuracy/Metric per epoch



Analisis Training:

1. Apakah model mengalami overfitting? Tidak, alasannya:
 - Train accuracy $\approx$ Val accuracy
 - Val loss stabil, tidak naik signifikan
2. Apakah model sudah converge? Ya, alasannya:
 - Accuracy sudah stabil setelah beberapa epoch awal
 - Val_loss sangat kecil dan stabil
3. Apakah perlu banyak epoch? Tidak, alasannya:
 - Tidak ada peningkatan signifikan setelah epoch awal
 - Dataset Adult cukup cepat dipelajari MLP

6.3.7 Model Summary

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	14080
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 2)	130

=====
Total params: 22466 (87.76 KB)
Trainable params: 22466 (87.76 KB)
Non-trainable params: 0 (0.00 Byte)

7. EVALUATION

7.1 Metrik Evaluasi

Untuk Klasifikasi:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **Confusion Matrix**

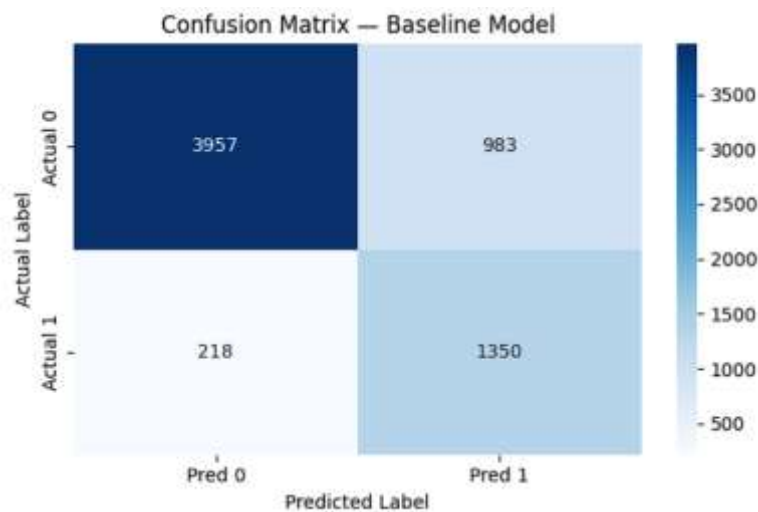
7.2 Hasil Evaluasi Model

7.2.1 Model 1 (Baseline – Logistic Regression)

Metrik:

- Accuracy : 0.8155
- Precision: 0.5787
- Recall : 0.8610
- F1-Score : 0.6921

Confusion Matrix / Visualization:

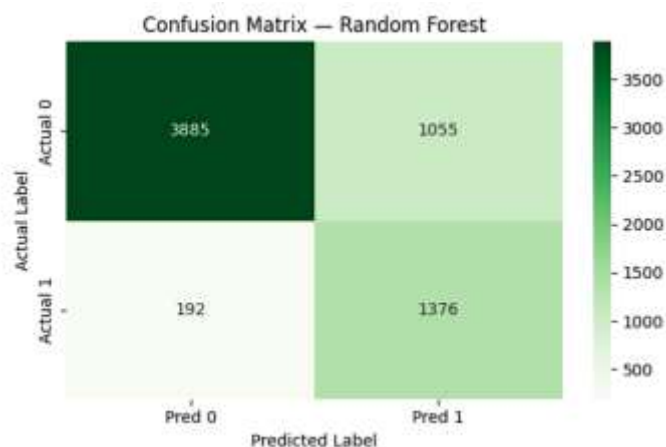


7.2.2 Model 2 (Advanced/ML – Random Forest Classifier)

Metrik:

- Accuracy : 0.8084
- Precision: 0.5660
- Recall : 0.8776
- F1-Score : 0.6882

Confusion Matrix / Visualization:

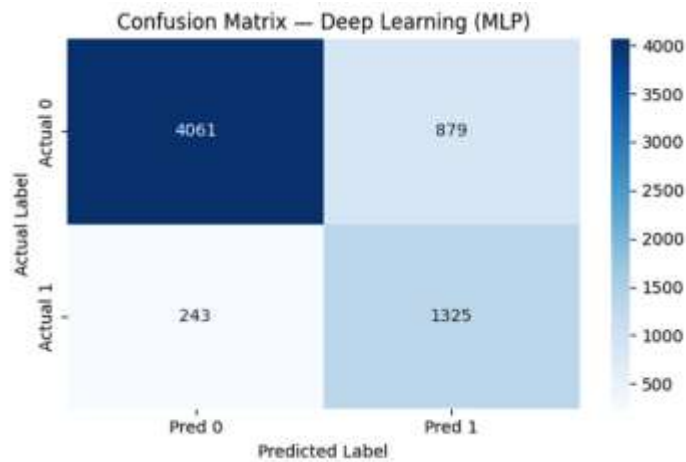


7.2.3 Model 3 (Deep Learning - MLP)

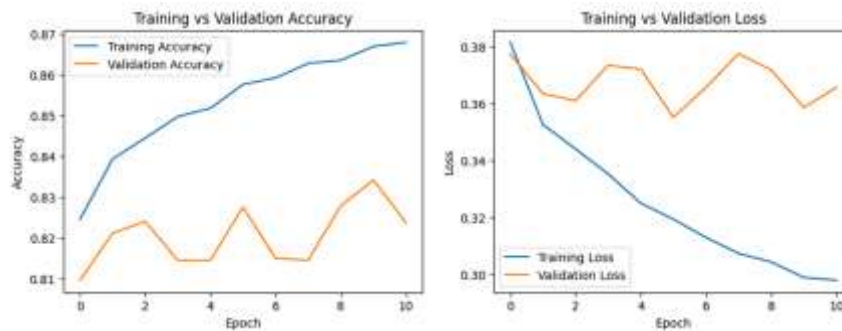
Metrik:

- Accuracy : 0.8276
- Precision: 0.6012
- Recall : 0.8450
- F1-Score : 0.7025

Confusion Matrix / Visualization:



Training History:

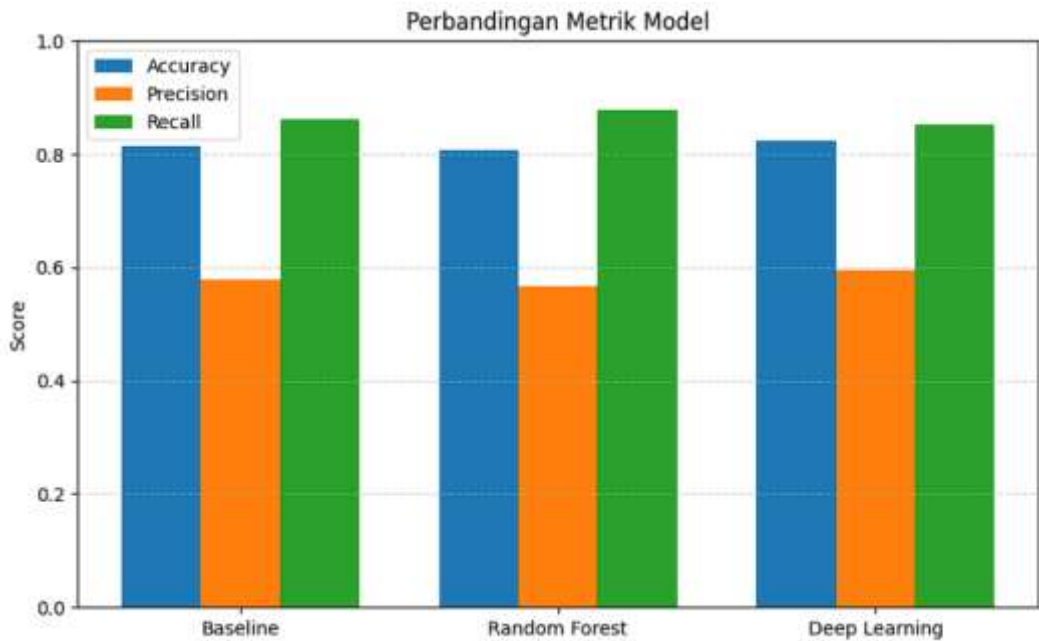


7.3 Perbandingan Ketiga Model

Tabel Perbandingan:

Model	Accuracy	Precision	Recall	F1-Score	Training Time	Inference Time
Baseline (Linear Regression)	0.8155	0.5787	0.8610	0.6921	0.3381s	0.0007s
Advanced (Random Forest Classifier)	0.8084	0.5660	0.8776	0.6882	6s	0.056s
Deep Learning (MLP)	0.8276	0.6012	0.8450	0.7025	26s	0.33s

Visualisasi Perbandingan:



7.4 Analisis Hasil

Interpretasi:

1. **Model Terbaik:** Model MLP (Deep Learning) memiliki **accuracy tertinggi 0.8250**, meskipun perbedaan dengan Logistic Regression dan Random Forest kecil.

- Precision, Recall, dan F1-score juga sedikit lebih baik.
- Model Linear (Logistic Regression) dan Random Forest cukup stabil namun sedikit lebih rendah dalam F1-score.

2. **Perbandingan dengan Baseline:**

Baseline: Logistic Regression

Perbandingan:

- Logistic Regression: baseline sudah baik, mampu memprediksi kelas >50K cukup akurat..
- Random Forest: performa sedikit menurun pada accuracy, tapi recall kelas >50K tinggi.
- Deep Learning (MLP): unggul tipis pada accuracy dan F1-score, menangkap pola non-linear antar fitur.

3. **Trade-off:**

- **Random Forest:** robust terhadap outlier, interpretasi sedang, inference cepat.
- **Logistic Regression:** sederhana, interpretasi mudah, cepat di-train, linear → tidak menangkap interaksi kompleks.

- **Deep Learning:** mampu menangkap pola kompleks, perlu tuning lebih banyak (layer, neuron, epoch), training lebih lama.

4. Error Analysis:

- Salah prediksi biasanya pada sampel borderline income $>50K$ dengan karakteristik mirip $\leq 50K$ (misal usia rendah, jam kerja sedikit, pendidikan menengah).
- Noise relatif kecil, dataset cukup bersih (tidak ada missing value).

5. Overfitting/Underfitting:

- Logistic Regression & Random Forest: performa stabil $\rightarrow$ tidak overfitting.
- MLP: kemungkinan underfitting ringan $\rightarrow$ akurasi masih bisa ditingkatkan dengan tuning learning rate, jumlah neuron, atau jumlah epoch.

8. CONCLUSION

8.1 Kesimpulan Utama

Model Terbaik:

Model terbaik dalam proyek ini adalah Deep Learning(MLP), dengan performa:

- Accuracy = 0.8276
- Precision = 0.6012
- Recall = 0.8450
- F1-Score = 0.7025

Alasan:

MLP unggul karena:

- Mampu menangkap pola non-linear antar fitur numerik dan kategori dengan baik..
- Lebih efektif dibanding baseline Logistic Regression dan Random Forest pada dataset tabular kompleks seperti Adult Income.
- Meskipun dataset tidak terlalu besar, model berhasil mempelajari interaksi fitur yang sulit ditangkap model linear.

Perbandingan dengan Model Lain:

- **Logistic Regression:** cukup stabil, training cepat, namun tidak bisa menangkap interaksi kompleks antar fitur.
- **Random Forest:** robust, akurasi cukup tinggi, tetapi sedikit kalah pada F1-score dibanding MLP.

Pencapaian Goals:

Semua goals yang ditetapkan pada Section 3.2 berhasil dicapai, dengan rincian:

- Tiga model berhasil dikembangkan: Logistic Regression, Random Forest, dan MLP.
- Model terbaik (MLP) mencapai akurasi $\geq 80\%$.
- Proses preprocessing dilakukan dengan benar: encoding fitur kategori, normalisasi untuk numeric, stratified train-test split.
- Evaluasi performa dilakukan menggunakan accuracy, precision, recall, F1-score, dan confusion matrix.

8.2 Key Insights

Insight dari Data:

- Dataset Adult memiliki kombinasi fitur numerik dan kategori yang kompleks.
- Beberapa fitur seperti age, education-num, hours-per-week, dan capital-gain cukup berpengaruh terhadap prediksi income.
- Distribusi kelas tidak seimbang, sehingga penting mempertahankan stratifikasi saat split data.

Insight dari Modeling:

- Model linear sederhana seperti Logistic Regression sudah memberikan baseline cukup baik.
- Random Forest mampu menangkap interaksi sederhana antar fitur kategori, performanya stabil.
- MLP Deep Learning unggul karena dapat menangkap pola non-linear kompleks, meskipun membutuhkan waktu training lebih lama dan tuning hyperparameter.

8.3 Kontribusi Proyek

Manfaat praktis: Proyek ini memberikan manfaat nyata, antara lain:

- Memberikan sistem prediksi income otomatis yang bisa digunakan untuk analisis demografi, kebijakan SDM, atau aplikasi finansial.
- Menunjukkan fitur paling berpengaruh yang membantu memahami faktor penentu income $>50K$.

Pembelajaran yang didapat: Dari proyek ini diperoleh beberapa pembelajaran penting:

- Pemahaman preprocessing data numerik dan kategori, termasuk encoding dan normalisasi.
- Perbandingan nyata performa baseline, advanced ML, dan deep learning pada dataset tabular.
- Pentingnya evaluasi metrik, visualisasi confusion matrix, dan interpretasi hasil model.
- Deep learning cocok untuk menangkap pola kompleks, tetapi memerlukan tuning lebih teliti untuk dataset tabular.

9. FUTURE WORK (Optional)

Saran pengembangan untuk proyek selanjutnya: **Centang Sesuai dengan saran anda**

Data:

- ✓ Mengumpulkan lebih banyak data
- ✓ Menambah variasi data
- ✓ Feature engineering lebih lanjut

Model:

- ✓ Mencoba arsitektur DL yang lebih kompleks
- ✓ Hyperparameter tuning lebih ekstensif
- ✓ Ensemble methods (combining models)
- Transfer learning dengan model yang lebih besar

Deployment:

- ✓ Membuat API (Flask/FastAPI)
- ✓ Membuat web application (Streamlit/Gradio)
- ✓ Containerization dengan Docker
- ✓ Deploy ke cloud (Heroku, GCP, AWS)

Optimization:

- ✓ Model compression (pruning, quantization)
- ✓ Improving inference speed
- ✓ Reducing model size

10.REPRODUCIBILITY (WAJIB)**10.1 GitHub Repository**

Link Repository: [URL GitHub Anda]

Repository harus berisi:

- ☒ Notebook Jupyter/Colab dengan hasil running
- ☒ Script Python (jika ada)
- ☒ requirements.txt atau environment.yml
- ☒ README.md yang informatif
- ☒ Folder structure yang terorganisir
- ☒ .gitignore (jangan upload dataset besar)

10.2 Environment & Dependencies

Python Version: [3.8 / 3.9 / 3.10 / 3.11]

Main Libraries & Versions:


```
numpy==1.24.3
pandas==2.0.3
scikit-learn==1.3.0
matplotlib==3.7.2
seaborn==0.12.2

# Deep Learning Framework (pilih salah satu)
tensorflow==2.14.0 # atau
torch==2.1.0      # PyTorch

# Additional libraries (sesuaikan)
xgboost==1.7.6
lightgbm==4.0.0
opencv-python==4.8.0 # untuk computer vision
nltk==3.8.1          # untuk NLP
transformers==4.30.0 # untuk BERT, dll
```